Semántica de MiniJava – Documento completo

Declaraciones, expresiones y sentencias

Borrador – Versión 3.0

Compiladores e Intérpretes – 2026

5 de octubre de 2026

1. Alcance

Este documento define los controles semánticos de MiniJava correspondientes a las Etapas 3 y 4. En la Etapa 3 se construye y consolida la tabla de símbolos y se chequean las declaraciones. En la Etapa 4 se construyen los árboles de sintaxis abstracta (AST) y se chequean los cuerpos de métodos y constructores. La generación de código y el control del punto de entrada corresponden a la Etapa 5.

Las reglas se aplican sobre programas sintácticamente válidos según las *Reglas de Sintaxis de MiniJava 2026*. Las secciones de declaraciones retoman la *Semántica de MiniJava – Parte I: Declaraciones* de 2026, con las aclaraciones de genericidad y contexto estático definidas en este documento.

Una *unidad* es un método o un constructor. El término *tipo declarado* comprende clases e interfaces; no se usa *clase* como sinónimo de interfaz.

Regla de oro Todo control o comportamiento que no esté especificado en este documento sigue las reglas de Java.

2. Tabla de símbolos

El compilador debe mantener una tabla global de tipos. Esa tabla contiene las clases e interfaces declaradas por el programa y las entidades predefinidas.

Cada clase debe registrar: nombre, parámetro genérico opcional, tipo heredado o interfaz implementada, atributos, métodos y constructores. Cada interfaz debe registrar: nombre, parámetro genérico opcional, interfaz extendida y métodos.

Cada método debe registrar: nombre, condición de estático, tipo de retorno y parámetros ordenados. Cada constructor debe registrar sus parámetros ordenados. Cada parámetro debe registrar nombre, tipo y posición.

Todas las entidades deben conservar el token de su declaración. La identidad de un método o constructor incluye su aridad, es decir, la cantidad de parámetros formales.

En la Etapa 4, cada método y constructor declarado tiene asociado el AST de su cuerpo. El análisis debe disponer de la clase, la unidad y el bloque actuales, y de las variables locales visibles en cada punto. Las variables locales registran, como mínimo, nombre, tipo, token de declaración y bloque al que pertenecen.

3. Tipos

Los tipos son `boolean`, `char`, `int`, tipos referencia, parámetros de tipo y arreglos. `void` sólo se utiliza como retorno de un método.

Un arreglo puede tener cualquier cantidad de dimensiones. Su tipo base puede ser primitivo, referencia o parámetro de tipo. Por ejemplo, `int[][]`, `String[]` y `T[]` son tipos de arreglos. Las dimensiones forman parte del tipo; los tamaños no.

Un tipo referencia debe nombrar una clase o interfaz existente, o una entidad predefinida. Si la entidad no es genérica, no puede suministrarse un argumento. Si es genérica, debe suministrarse un argumento válido, salvo la excepción de herencia sin argumento de la Sección 5.

El argumento debe ser un parámetro de tipo válido en el contexto actual o el nombre de una clase o interfaz existente no genérica. No puede utilizarse el nombre de otra entidad genérica omitiendo su argumento como modo de introducir un tipo raw. La gramática no permite escribir instanciaciones anidadas.

Un parámetro de tipo sólo puede escribirse en los contextos de instancia de la clase o interfaz que lo declara. Por ejemplo:

```
class Caja<T> {
    T contenido;
    T obtener() { return contenido; }
}
```

La conservación de parámetros en miembros heredados no habilita escribirlos en nuevas declaraciones fuera de su ámbito. Su uso en contextos estáticos se rechaza según la Sección 10.4.

Los identificadores de parámetros de tipo son una única letra mayúscula. Los nombres de clases e interfaces tienen una letra mayúscula inicial y al menos un carácter adicional, según la especificación léxica de 2026.

La gramática admite un único parámetro por clase o interfaz y un único argumento genérico por uso. No admite argumentos primitivos, argumentos arreglo, argumentos genéricos anidados, comodines, cotas explícitas ni el operador diamante. Estas restricciones son sintácticas.

El literal `null` tiene un tipo especial que no puede escribirse en una declaración. `void` no es un tipo de valor: una llamada a un método `void` no produce un valor utilizable.

4. Clases e interfaces

Clases e interfaces comparten la tabla global de tipos. Cada nombre de tipo puede tener una sola declaración. Tampoco pueden redeclararse `Object`, `String` ni `System`.

Una clase con `extends R` debe extender una clase existente `R`. Una clase con `implements R` debe implementar una interfaz existente `R`. Una clase sin relación explícita extiende `Object`.

Una interfaz con `extends R` debe extender una interfaz existente `R`. Las cadenas de herencia entre clases y de extensión entre interfaces deben ser acíclicas.

La gramática permite una única relación explícita por clase: `extends` o `implements`. No permite combinar ambas cláusulas ni extender o implementar varios tipos. Una clase que implementa una interfaz también es un subtipo de `Object`, como las demás clases.

Todos los tipos declarados son globalmente visibles. Su uso no depende de que la declaración aparezca antes o después en el archivo. Las interfaces no tienen atributos, constructores ni cuerpos de métodos en la gramática base.

5. Genericidad y herencia

Una clase o interfaz declara, como máximo, un parámetro de tipo. Ese parámetro puede utilizarse en los tipos de sus atributos, retornos, parámetros formales y expresiones de sus cuerpos de instancia, respetando la gramática y las restricciones de contexto.

Al heredar de una clase o interfaz genérica con argumento explícito, los tipos paramétricos de los miembros heredados se instancian con ese argumento.

Al heredar de una clase o interfaz genérica sin argumento explícito, se conserva el nombre del parámetro de tipo del tipo heredado. La consolidación no lo elimina ni lo reemplaza por un tipo concreto.

```
class Base<T> {
    T obtener() { return null; }
}
class Derivada extends Base {}
```

Luego de consolidar `Derivada`, el método heredado `obtener` conserva retorno `T`. Esta regla también se aplica a `implements` y a la extensión de interfaces.

5.1. Instanciación en accesos y llamadas

La Etapa 4 utiliza los tipos de los miembros vistos desde el tipo estático del receptor. Si el receptor tiene tipo `Caja<String>`, un atributo declarado `T` se ve como `String`; un parámetro formal `T` se chequea como `String`; y un retorno `T[]` se ve como `String[]`.

La sustitución se aplica a todas las apariciones del parámetro en el tipo, incluidos arreglos y argumentos genéricos. En una cadena de herencia se componen las sustituciones de cada relación:

```
class Base<T> {
    T dato;
    T obtener() { return dato; }
}
class Intermedia<U> extends Base<U> {}
class Final extends Intermedia<String> {}
```

En `Intermedia<U>`, el atributo y el retorno heredados tienen tipo `U`. En `Final`, ambos tienen tipo `String`.

Estas vistas no deben modificar las declaraciones originales ni afectar a otros usos del mismo tipo genérico. El cuerpo declarado en `Base<T>` se chequea en su contexto de declaración, no como si hubiese sido escrito dentro de cada descendiente.

Si una relación de herencia no proporciona argumento, los accesos a los miembros heredados conservan el parámetro registrado al consolidar. En el primer ejemplo, una llamada a `obtener()` mediante un receptor `Derivada` tiene tipo paramétrico `T`, no `Object`.

Conservar un parámetro heredado no equivale a declararlo en el descendiente. No se renombra automáticamente con el nombre de un parámetro propio del descendiente. Para representar los tipos se debe conservar su declaración de origen, además del nombre.

5.2. Ausencia de tipos raw

MiniJava no incorpora tipos raw. Fuera de las cláusulas de herencia o implementación, usar una entidad genérica sin argumento es un error semántico. Esto incluye atributos, parámetros,

retornos, tipos base de arreglos y creaciones de objetos.

```
class Caja<T> {
    T contenido;
}
class Uso {
    Caja dato;                    // error: falta argumento
    Caja<String> texto;           // válido
    void ejemplo() {
        var caja = new Caja();    // error: falta argumento
        var otra = new Caja<String>(); // válido
    }
}
```

La excepción de herencia sin argumento no borra tipos paramétricos ni habilita conversiones unchecked. Tampoco autoriza a omitir argumentos en los demás usos de la entidad genérica.

6. Miembros

6.1. Atributos

Un atributo debe tener un tipo válido. Una clase no puede declarar dos atributos con el mismo nombre. Una clase tampoco puede declarar un atributo con el mismo nombre que un atributo heredado.

Todos los atributos de la gramática base son de instancia y se consideran públicos para los accesos entre objetos. No se incorporan modificadores de visibilidad ausentes de la gramática.

6.2. Métodos

Un método debe tener tipo de retorno válido y parámetros de tipos válidos. Los métodos estáticos deben respetar además las restricciones sobre parámetros de tipo de la Sección 10.4.

MiniJava permite sobrecarga simple. Dos métodos de una misma clase o interfaz pueden tener el mismo nombre si poseen distinta aridad. La clave de un método es `(nombre, aridad)`.

Dos métodos de una misma clase o interfaz con igual nombre e igual aridad son una declaración repetida, aunque difieran en los tipos o nombres de sus parámetros.

```
void mostrar() {}
void mostrar(int n) {}     // válido: mostrar/0 y mostrar/1
void imprimir(int n) {}
void imprimir(char c) {}   // error: imprimir/1 repetido
```

Si un método de instancia tiene el mismo nombre y aridad que un método de instancia heredado, lo redefine. Ambas signaturas deben ser exactamente iguales: misma cantidad de parámetros, mismos tipos de parámetros en el mismo orden y mismo tipo de retorno. Los nombres de parámetros no forman parte de la signatura. La comparación se realiza luego de aplicar la instanciación genérica correspondiente.

Los métodos estáticos no participan de la redefinición. Si un ancestro declara un método estático con clave `(nombre, aridad)`, ningún descendiente puede declarar otro método con esa clave, sea

estático o de instancia. Tampoco puede declararse un método estático con la misma clave que un método de instancia heredado.

Una clase que implementa una interfaz debe poseer, directa o heredadamente, un método de instancia compatible para cada método de la interfaz. Un método estático no implementa un método de interfaz. Los métodos se consideran públicos; las interfaces no admiten métodos estáticos en la gramática base.

6.3. Constructores

Un constructor debe tener el mismo nombre que su clase. Una clase puede tener varios constructores con distinta aridad. Dos constructores de una misma clase con igual aridad son una declaración repetida, aunque difieran en los tipos de sus parámetros.

Los parámetros de un constructor deben tener tipos válidos y nombres no repetidos. Si una clase no declara constructores, el compilador debe agregar un constructor por defecto sin parámetros y con cuerpo vacío. Si declara al menos un constructor, no se agrega el constructor por defecto.

Los constructores no se heredan. Las interfaces no tienen constructores. La gramática no permite invocaciones explícitas `super(...)` o `this(...)`.

MiniJava no invoca implícitamente a un constructor de la superclase. No se exige que el ancestro directo tenga un constructor sin parámetros ni se agrega una llamada a él en un constructor declarado o generado por defecto.

```
class Base {
    public Base(int valor) {}
}
class Derivada extends Base {
    public Derivada() {} // válido: no hay llamada implícita a Base()
}
```

6.4. Parámetros formales

Los parámetros de un método o constructor están ordenados. En una misma unidad no pueden repetirse nombres de parámetros. Cada parámetro debe tener un tipo válido.

7. Entidades predefinidas

Antes de analizar el programa, la tabla global debe contener `Object`, `String` y `System`.

Object Declara:

```
static void debugPrint(int i)
String toString()
```

`debugPrint` imprime un entero y finaliza la línea. `toString` es un método de instancia y su retorno es `String`.

String Es el tipo de los literales de cadena, hereda de `Object` y no tiene métodos propios. Dispone del método heredado `toString()`.

System Hereda de `Object` y declara:

```
static int read()
static void printB(boolean b)
static void printC(char c)
static void printI(int i)
static void printS(String s)
static void println()
static void printBln(boolean b)
static void printCln(char c)
static void printIln(int i)
static void printSln(String s)
```

`read` lee el próximo byte de la entrada estándar. Los métodos `print` imprimen el valor indicado; las variantes terminadas en `ln` agregan un separador de línea. `println` imprime únicamente ese separador.

Los nombres son parte de la especificación. No se justifican mediante una prohibición de sobrecarga por nombre: en 2026 existe sobrecarga por distinta aridad.

8. Procesamiento de declaraciones

En una primera pasada se construyen las tablas de símbolos. Se controlan nombres repetidos de tipos, atributos, métodos con igual nombre y aridad, constructores con igual aridad y parámetros de una misma unidad.

En una segunda pasada se controlan tipos válidos, uso completo de entidades genéricas fuera de la excepción de herencia, restricciones genéricas en encabezados estáticos, relaciones entre clases e interfaces, circularidad, redefiniciones y contratos de interfaces.

Finalmente se consolidan clases e interfaces. La consolidación incorpora miembros heredados, respeta redefiniciones válidas e instancia o conserva parámetros según la relación de herencia.

El chequeo de cuerpos se realiza después de estos controles. No se controla la existencia ni la unicidad de un punto de entrada en las Etapas 3 y 4.

9. Conformidad de tipos

Un tipo origen conforma con un tipo destino cuando un valor del primero puede utilizarse donde se exige el segundo. La dirección es *origen hacia destino*: en una asignación, el tipo del lado derecho conforma con el del izquierdo; en una llamada, el argumento actual conforma con el parámetro formal; en un retorno, la expresión conforma con el retorno declarado.

9.1. Tipos primitivos

Cada tipo primitivo conforma únicamente consigo mismo. No se realizan conversiones implícitas entre `char` e `int`, ni entre primitivas y referencias. No hay boxing ni unboxing.

9.2. Clases e interfaces

Todo tipo conforma consigo mismo. Una clase conforma con sus clases ancestras y con las interfaces que implementa, incluyendo las interfaces ancestras de estas últimas. Una interfaz

conforma con las interfaces que extiende, directa o indirectamente. Los tipos de clases e interfaces conforman con `Object`.

El subtipado es transitivo. Un subtipo puede utilizarse donde se espera un supertipo; no se permite automáticamente la dirección inversa. La conformidad con `Object` no se extiende a parámetros de tipo no instanciados.

9.3. Tipos parametrizados

La relación de herencia se recorre aplicando las sustituciones correspondientes. Una clase que implementa `Contenedor<String>` conforma con `Contenedor<String>`, pero no por ese motivo con `Contenedor<Object>`.

Los argumentos genéricos son invariantes: aunque `String` conforme con `Object`, `Caja<String>` no conforma con `Caja<Object>`. La conformidad no puede decidirse comparando sólo el nombre `Caja` e ignorando su argumento. La invariancia no elimina relaciones de herencia declaradas entre entidades; impide inducir conformidad por la herencia entre argumentos.

Los parámetros de tipo no tienen cotas. No se les asigna la cota implícita `Object` de Java. Un parámetro no instanciado `T` no conforma con `Object`, ni `Object` conforma con `T`.

Mientras no haya sido sustituido, un parámetro de tipo sólo conforma consigo mismo. El tratamiento especial de `null` se indica más adelante. Un valor `String` no conforma con un destino `T` en un cuerpo chequeado paramétricamente, ni `T` conforma con `String`.

```
class Caja<T> {
    T contenido;
    T obtener() { return contenido; }         // válido
    Object comoObjeto() { return contenido; } // error: T -> Object
    void guardarObjeto(Object objeto) {
        contenido = objeto;                  // error: Object -> T
    }
}
```

Cuando una vista de un miembro sustituye `T` por un tipo concreto, se aplica la conformidad de ese tipo. Si el retorno de `obtener()` se ve como `String` mediante `Caja<String>`, el valor obtenido conforma con `Object`. Esto no implica conformidad del `T` no instanciado con `Object`.

9.4. Arreglos

Los arreglos son invariantes. Un tipo arreglo conforma con otro únicamente si son exactamente iguales: mismo tipo base, incluidos sus argumentos genéricos o el parámetro correspondiente, y misma cantidad de dimensiones.

La herencia entre tipos base no induce conformidad entre arreglos:

```
String[]  -> String[]            // conforma
String[]  -> Object[]            // no conforma
int[][]   -> Object[]            // no conforma
int[][]   -> int[]               // no conforma
Caja<String>[] -> Caja<Object>[] // no conforma
T[]       -> Object[]            // no conforma
```

Todo tipo arreglo conforma con `Object`. Esto no habilita conversiones entre arreglos de tipos distintos ni permite indexar una expresión cuyo tipo estático sea `Object`.

```
void recibir(Object valor) {}
void ejemplo() {
    recibir(new int[10]); // válido: int[] conforma con Object
}
```

Una indexación puede producir un subarreglo. Su asignación exige la misma conformidad invariante: si `matriz` tiene tipo `String[][]`, `matriz[0]` tiene tipo `String[]` y no puede asignarse a un destino `Object[]`.

No se requiere un control dinámico de compatibilidad al almacenar componentes para compensar conversiones covariantes, porque esas conversiones no se admiten. Esto no elimina los controles de rango o referencia nula que se definan para ejecución.

9.5. Null y void

El tipo especial de `null` conforma con clases, interfaces, parámetros de tipo y arreglos. No conforma con primitivas. No determina por sí mismo el tipo de una variable local declarada con `var`. Esta conformidad especial no introduce una cota `Object` para parámetros.

`void` no conforma con ningún tipo de valor. Una llamada `void` no puede ser operando, argumento actual, índice, tamaño, inicializador, lado de asignación ni expresión de un retorno con valor.

10. Nombres y contextos

10.1. Variables y atributos

Para resolver un identificador utilizado como variable sin receptor explícito, se busca en este orden:

1. Una variable local visible, desde el bloque actual hacia los contenedores.
2. Un parámetro de la unidad actual.
3. Un atributo de la clase actual, incluidos los heredados.

Si no se encuentra una entidad, el acceso es incorrecto. Si se selecciona un atributo y el contexto es estático, el acceso implícito es incorrecto. Una local o un parámetro puede ocultar un atributo; `this.nombre` selecciona el atributo en contexto de instancia.

Para `e.nombre`, se chequea `e` y se obtiene su tipo estático. El atributo se busca en ese tipo, no en la clase que contiene la sentencia ni en la clase dinámica del objeto. Se aplican las sustituciones genéricas pertinentes.

Las interfaces no declaran atributos. Primitivos, `void`, el tipo especial de `null` y parámetros no instanciados no permiten seleccionar atributos por punto. Los arreglos admiten `length` según la Sección 11.8.

10.2. Métodos

Para `m(e1,...,en)`, se busca `(m,n)` en los métodos de la clase actual, incluidos los heredados. En un método estático, una llamada sin receptor explícito sólo puede seleccionar un método estático.

Para `e.m(e1,...,en)`, se chequea el receptor y se busca `(m,n)` entre los métodos de su tipo estático. Las interfaces permiten seleccionar sus métodos declarados y heredados. Se permite seleccionar tanto un método de instancia como uno estático mediante una expresión receptora, respetando las restricciones estructurales de las sentencias.

Un parámetro no instanciado no tiene cotas ni garantiza miembros de `Object`. No se permite resolver métodos mediante un receptor `T`: `contenido.toString()` es incorrecto si `contenido` tiene tipo `T`. Esto no impide seleccionar miembros de un receptor de tipo conocido `Caja<T>`, ni de uno cuyo tipo paramétrico ya fue sustituido por un tipo concreto.

Para `Clase.m(e1,...,en)`, `Clase` debe nombrar una clase existente y el método debe ser estático. Un método de instancia no puede invocarse usando sólo el nombre de una clase.

Si `Persona.consultar` es estático y `persona` tiene tipo `Persona`, se permiten tanto `Persona.consultar(...)` como `persona.consultar(...)`. La expresión receptora se chequea y se evalúa; la selección del estático no usa despacho dinámico.

La resolución por aridad selecciona, como máximo, un método. Después se controlan tipos de argumentos. No se busca otra variante por tipos si esos argumentos no conforman.

10.3. Constructores

Para `new Clase(e1,...,en)`, se valida el tipo referencia y se exige una clase, no una interfaz. Si es genérica, debe tener argumento. Se selecciona el constructor con aridad `n`, sin buscar en ancestros ni seleccionar por tipos de argumentos.

Clases, métodos y constructores pueden resolverse aunque sus declaraciones aparezcan después. Las locales sólo son visibles después de su declaración, según la Sección 12.3.

10.4. Contexto estático

En un método estático no se permite `this`, un acceso implícito a un atributo ni una llamada implícita a un método de instancia. La restricción también se aplica dentro de expresiones parentizadas o subexpresiones.

Sí se permite acceder a miembros de instancia mediante un objeto obtenido de un parámetro, una local, una llamada o una creación:

```
class Persona {
    int edad;
    int obtenerEdad() { return edad; }
    static int consultar(Persona persona) {
        return persona.obtenerEdad();
    }
}
```

El receptor explícito `persona` no es el receptor implícito de la unidad estática.

Se rechaza el uso del parámetro de tipo de la clase en encabezados y cuerpos de métodos estáticos. La prohibición comprende su aparición directa y dentro de tipos de arreglo o argumentos genéricos. No hay métodos con parámetros de tipo propios en la gramática.

```
class Caja<T> {
    static T identidad(T valor) { return valor; } // error
    static void crear() {
        var datos = new T[10];                    // error
    }
    static void valido() {
        var datos = new Caja<String>[10];         // válido
    }
}
```

11. Expresiones

Cada expresión chequea sus subexpresiones antes de determinar su resultado. El análisis distingue tipo resultante, condición de destino asignable y forma admisible como sentencia de llamada. Tener un tipo correcto no basta para satisfacer las otras propiedades.

El AST respeta la estructura determinada por la gramática y el análisis sintáctico de 2026. El chequeo semántico no introduce la precedencia de Java ni reorganiza operadores. La gramática de `ExpresionCompuesta` presenta los binarios en un mismo nivel; la política de desambiguación es la fijada para Etapa 2.

11.1. Literales

Los literales son correctamente tipados por sí mismos:

Literal	Tipo
Entero	`int`
Carácter	`char`
`true`, `false`	`boolean`
Cadena	`String`
`null`	Tipo especial de `null`

Un literal no es asignable. Los miembros de una cadena se chequean mediante un receptor `String`.

11.2. Operadores

Las subexpresiones deben producir valores de los tipos indicados:

Operador	Tipos de operandos	Resultado
Unarios `+`, `-`	`int`	`int`
Unario `!`	`boolean`	`boolean`
`+`, `-`, `*`, `/`, `%`	`int`, `int`	`int`
`<`, `<=`, `>`, `>=`	`int`, `int`	`boolean`
`&&`, `||`	`boolean`, `boolean`	`boolean`
`==`, `!=`	Conformantes en alguna dirección	`boolean`

En igualdad y desigualdad, el tipo de un operando conforma con el del otro, o viceversa. Dos `char` pueden compararse; `char` e `int` no. Las referencias se comparan por identidad, no por contenido, también en cadenas y arreglos.

No hay promoción numérica implícita ni concatenación de cadenas mediante `+`. Un resultado `void` no se compara ni se utiliza con operadores. `&&` y `||` se evalúan con cortocircuito; el chequeo analiza ambos operandos aunque uno pueda no ejecutarse.

11.3. This

`this` es el receptor implícito y sólo es válido en constructores y métodos de instancia. Su tipo es la clase actual con su parámetro si corresponde: dentro de `Caja<T>`, tiene tipo `Caja<T>`. Por sí solo no es asignable.

11.4. Paréntesis

Una expresión parentizada `(e)` conserva únicamente el tipo de `e`, no sus características estructurales. No hereda la condición de destino asignable ni la clasificación como asignación o llamada. Por ejemplo, `(x)` tiene el tipo de `x`, pero no es asignable; `(metodo())` conserva el tipo del resultado, pero no cuenta como llamada para validar una sentencia.

Por lo tanto, `(x) = 3;` y `(x = 3);` se rechazan como sentencias. También se rechazan `(metodo());` y `(objeto).metodo();`: no se usa un primario parentizado como base de una sentencia de llamada o de un destino de asignación.

Los paréntesis sí pueden agrupar valores en contextos de expresión: `var total = (a + b) * c;` es válido cuando los operandos tienen los tipos adecuados. También puede usarse `var resultado = (x = 3);` si la asignación interna es correcta; el resultado parentizado tiene tipo `int`, pero no es asignable ni se clasifica como sentencia de asignación.

Las restricciones estructurales no dispensan de chequear la expresión interna. En particular, los paréntesis no hacen válido un acceso a `this` en contexto estático ni convierten una llamada `void` en un valor.

11.5. Accesos a variables

Locales, parámetros y atributos se resuelven según la Sección 10. El tipo es el de la entidad seleccionada, con las sustituciones pertinentes. Sus accesos directos son asignables.

La asignabilidad de un encadenado depende de su forma completa y último acceso, respetando la restricción sobre primarios parentizados. Una llamada no es asignable, pero `crearPersona().edad` puede serlo. Crear un arreglo no es asignable por sí solo, pero acceder a un componente sí puede serlo.

11.6. Llamadas a métodos

Luego de resolver por nombre y aridad, cada argumento debe estar correctamente tipado y producir un valor. Su tipo conforma con el formal de la misma posición.

Parámetros y retorno se interpretan desde el receptor con las sustituciones genéricas. El resultado es el retorno del método seleccionado. Si es `void`, la llamada no produce valor y sólo se utiliza donde se admita una llamada sin resultado.

La selección semántica usa el tipo estático. En un método de instancia, la implementación ejecutada puede corresponder a una redefinición en la clase dinámica del objeto. Esto no modifica el chequeo de tipos. Un estático no usa ese despacho dinámico.

11.7. Creación de objetos

`new Clase<Argumento>(e1,...,en)` utiliza un tipo referencia válido que nombre una clase. Se selecciona el constructor por aridad y se chequean argumentos con las sustituciones pertinentes. Si la clase es genérica, no puede omitirse el argumento.

El resultado tiene el tipo completo del objeto: `new Caja<String>()` tiene tipo `Caja<String>`. El constructor no declara retorno; ese resultado pertenece a la creación. No se invoca un constructor de la superclase.

No se instancian interfaces. La gramática tampoco permite `new T()`, porque `T` es un parámetro, no un `idClase`. Crear un objeto no designa un destino asignable.

11.8. Arreglos

Creación En `new TipoBase[e1]...[en]`, cada tamaño es una expresión correctamente tipada de tipo `int`. El resultado conserva el tipo base completo y tiene `n` dimensiones. `new int[3][4]` tiene tipo `int[][]`.

El tipo base debe ser válido en el contexto. Puede ser primitivo, referencia, referencia genérica instanciada o parámetro de tipo. Si nombra una entidad genérica, no puede omitir su argumento. Puede nombrar una interfaz: `new Figura[5]` crea referencias `Figura`, no objetos que instancien esa interfaz.

Se permiten explícitamente arreglos con tipos base paramétricos o referencias genéricas instanciadas, sin exigir reificación. Dentro de una unidad de instancia de una clase que declara `T`, son válidos:

```
var elementos = new T[10];              // tipo T[]
var cajas = new Caja<String>[10];       // tipo Caja<String>[]
var matriz = new Caja<T>[3][4];         // tipo Caja<T>[][]
```

Crear el arreglo reserva posiciones para componentes; no crea objetos del tipo base ni invoca sus constructores. `new T[10]` no equivale a diez llamadas `new T()`.

Estas creaciones son independientes del diamante, que la gramática no incorpora. La invariancia impide almacenamientos incompatibles habilitados por conversiones covariantes. No se exige conservar argumentos genéricos o parámetros en la MV para ese control dinámico.

Cada dimensión escrita debe tener tamaño. No se admite `new int[3][]` ni inicializadores `{1,2,3}`. Estas restricciones son sintácticas.

Indexación En `e[i]`, `e` produce un arreglo e `i` tiene tipo `int`. Si `e` tiene tipo `Tipo[]`, el resultado tiene tipo `Tipo`. Cada indexación elimina una dimensión.

Un componente puede ser asignable, respetando la restricción sobre primarios parentizados. Si `matriz` tiene tipo `int[][]`, `matriz[0]` tiene tipo `int[]` y `matriz[0][1]` tiene tipo `int`. Una fila o un elemento pueden ser destinos con la conformidad correspondiente.

El encadenado puede continuar: si `personas` tiene tipo `Persona[]`, `personas[0].edad` usa `Persona` como receptor. Un primitivo no permite punto ni otra indexación. Un parámetro no instanciado no permite seleccionar miembros por punto.

Miembros Los arreglos tienen `length`, de tipo `int` y sólo lectura. Representa la cantidad de componentes de la dimensión del receptor. En un multidimensional, `matriz.length` es el tamaño de la primera dimensión y `matriz[0].length` es el tamaño de la fila seleccionada.

También disponen de `toString()` por `Object`, sin parámetros y con retorno `String`. No se incorpora `clone()` ni otros miembros de arreglos.

```
var datos = new int[10];
var cantidad = datos.length;   // tipo int
var texto = datos.toString(); // tipo String
// datos.length = 3;           // error: sólo lectura
// datos.clone();              // error: método no disponible
```

Si una referencia a un arreglo tiene tipo estático `Object`, sólo permite los miembros disponibles en `Object`; no permite indexación ni acceso a `length`.

Controles estáticos y ejecución Se verifican tipos de tamaños e índices, pero no se exige demostrar que tamaños sean no negativos ni que todo índice esté dentro del rango. Esas condiciones dependen en general de valores de ejecución. Los fallos por referencia nula, tamaño negativo o índice fuera de rango no son errores de tipado de Etapa 4. Aquí no se especifica cómo los reportará la MV.

11.9. Asignaciones

`izquierda = derecha` es correcta si ambas subexpresiones están correctamente tipadas, `izquierda` designa un destino asignable y el tipo de `derecha` conforma con el de `izquierda`.

Son asignables los accesos a locales, parámetros, atributos y componentes, salvo las restricciones estructurales de la Sección 11.4. No lo son literales, `this`, expresiones parentizadas, llamadas por sí solas, creaciones por sí solas, resultados de operadores ni `length`.

La asignación produce un valor del tipo del destino; su resultado no es a su vez asignable. Puede usarse en contextos de expresión admitidos: `return x = 3;` puede ser correcto si `x` y el retorno tienen tipo `int`.

`Expresion` contiene a lo sumo una asignación en ese nivel. `x = y = 3` no se admite sin paréntesis que introduzcan otra expresión; `x = (y = 3)` sí puede derivarse y es semánticamente válido si los tipos conforman. Los paréntesis de la derecha producen un valor, no un destino asignable.

El inicializador de `var` es `ExpresionCompuesta`, pero puede contener una asignación parentizada: `var z = (x = 3);`.

12. Sentencias

Las sentencias no tienen tipo. Su correctitud depende del chequeo de expresiones, forma y contexto.

12.1. Sentencia vacía

`;` es correcta por sí misma y no introduce declaraciones ni modifica el contexto.

12.2. Asignación y llamada

Una sentencia de asignación es correcta si su expresión principal es una asignación que satisface la Sección 11.9.

Una sentencia de llamada tiene como operación final una invocación a método o una creación de objeto mediante constructor. Debe estar correctamente chequeada; si produce valor, se descarta. Una llamada `void` es válida aquí.

Si hay encadenados, el último es una llamada a método. No basta con una llamada intermedia. Se rechaza una expresión principal parentizada y una sentencia de llamada basada en un primario parentizado, según la Sección 11.4.

```
persona.obtenerEdad();         // llamada: correcta
new Persona();                // creación de objeto: correcta
crearPersona().obtenerEdad(); // llamada: correcta
crearPersona().edad;          // incorrecta como sentencia
personas[0];                  // incorrecta como sentencia
```

```
1 + 2;                      // incorrecta como sentencia
new int[3];                 // incorrecta como sentencia
(x) = 3;                    // incorrecta: no asignable
(x = 3);                    // incorrecta como sentencia
(metodo());                 // incorrecta como sentencia
(objeto).metodo();          // incorrecta como sentencia
```

Crear un arreglo no se considera llamar a un constructor. La gramática reconoce expresiones seguidas de punto y coma, pero se rechazan las que no sean asignaciones ni llamadas válidas.

12.3. Variables locales

Una declaración `var nombre = e;` es correcta cuando:

- El nombre no coincide con un parámetro de la unidad actual.
- No coincide con una local ya declarada en el bloque actual o un contenedor donde siga visible.
- El inicializador está correctamente tipado y produce un valor distinto del tipo especial de `null`.

La variable recibe exactamente el tipo del inicializador, conservando parámetros o argumentos y dimensiones. Una llamada `void` no permite inferir un tipo.

Es visible después de declararse y hasta terminar su bloque. El inicializador se chequea antes de incorporarla a las variables visibles. No se habilita una referencia a sí misma en el inicializador; si el nombre ya selecciona un atributo, se resuelve según el contexto previo.

Una local puede ocultar un atributo, pero no un parámetro ni una local todavía visible de un contenedor. Bloques hermanos pueden declarar el mismo nombre porque sus ámbitos no se superponen.

```
void ejemplo(int dato) {
    var valores = new int[3];
    {
        var indice = 0;
        valores[indice] = dato;
    }
    {
        var indice = 1;       // válido: otro bloque
    }
    var vacio = null;         // error: no se infiere tipo
}
```

Si una expresión produce un parámetro conservado en un miembro heredado, `var` registra ese tipo sin declararlo como nuevo parámetro de la clase actual.

Se rechaza una declaración local como cuerpo directo de `if`, `else` o `while`. Para declarar una variable en esos cuerpos se requiere un bloque con llaves.

```
if (condicion) var x = 1;      // error semántico
while (condicion) var x = 1;   // error semántico
if (condicion) { var x = 1; }  // válido
```

12.4. If y while

En `if (e) s1`, con o sin `else s2`, la condición es una expresión correctamente tipada `boolean` y cada rama presente es correcta. En `while (e) s`, la condición es `boolean` y el cuerpo es correcto.

Referencias y enteros no se convierten a `boolean`. Una asignación puede ser condición si la gramática la permite y su resultado es `boolean`. Se chequean todas las ramas y el cuerpo aunque no se ejecuten, y se respeta la prohibición de declaraciones locales sin bloque.

12.5. Return

En un método `void`, sólo se permite `return;` sin expresión. En un método con retorno distinto de `void`, una sentencia de retorno debe ser `return e;` y el tipo de `e` conforma con el retorno declarado.

En un constructor se permite `return;` para finalizar su ejecución, pero no `return e;`. Que `new Clase(...)` produzca un objeto no significa que el constructor retorne explícitamente ese objeto.

Un retorno con expresión de resultado `void` es incorrecto, incluso en un método `void`.

MiniJava no exige la presencia de un retorno en un método con tipo de retorno distinto de `void`, ni controla retorno en todos los caminos. Un cuerpo vacío o un camino que finaliza sin devolver un valor no es un error semántico por ese motivo. Cada retorno que sí aparezca debe satisfacer las reglas anteriores.

No se rechazan sentencias por ser inalcanzables. Se las chequea igualmente: la inalcanzabilidad no dispensa de resolver nombres ni verificar tipos y contexto.

```
int sinRetorno() {}           // válido en MiniJava
int parcial(boolean elegir) {
    if (elegir) return 1;
}                            // válido: no se exige retorno total
int conCodigoInalcanzable() {
    return 1;
    var numero = 2;           // permitido y chequeado
}
```

Estas decisiones definen la aceptación semántica; no establecen aquí qué valor produce una ejecución que alcanza el final de un método sin devolverlo.

12.6. Bloques

Un bloque es correcto si todas sus sentencias son correctas. Se chequean en orden para respetar visibilidad local, incluso después de un retorno. Al ingresar se establece un ámbito; al finalizar dejan de estar visibles sus locales.

El bloque exterior dispone de parámetros desde el comienzo. Los anidados acceden a parámetros y locales visibles de contenedores. La salida restaura el contexto previo, sin dejar variables accesibles en bloques siguientes.

13. Otras Diferencias explícitas respecto de Java

Los tipos genéricos y los arreglos son invariantes. Los parámetros de tipo no tienen cotas, tampoco una cota implícita `Object`. MiniJava no incorpora tipos raw, borrado de miembros a `Object` ni

conversiones unchecked. La herencia genérica sin argumento es una excepción propia del lenguaje que conserva el parámetro original.

Se permite crear arreglos con tipos base paramétricos o referencias genéricas instanciadas sin exigir reificación. Los arreglos conforman con `Object` y ofrecen únicamente `length` y el método de instancia `toString()` de `Object`, además de la indexación.

Los constructores no invocan implícitamente a un constructor de la superclase. Las expresiones parentizadas conservan el tipo, pero no las características estructurales de la expresión contenida. No se exige que un método con retorno devuelva un valor en todos sus caminos ni se rechaza código inalcanzable. En las Etapas 3 y 4 no se exige `main`.

14. Procesamiento de cuerpos

Durante el análisis sintáctico se construyen AST de métodos y constructores, junto con la tabla de símbolos. Los AST conservan tokens para localizar errores.

Después del chequeo y consolidación de declaraciones se recorren los cuerpos. Cada cuerpo se chequea en su clase y unidad de origen. Incorporar un método heredado a una tabla consolidada no implica copiar ni volver a chequear su cuerpo en el descendiente.

Las expresiones propagan tipos y resuelven nombres. Los nodos de sentencias validan forma y contexto. Los bloques y declaraciones administran visibilidad. Estas tareas se realizan en conjunto: el tipo de un acceso depende de la entidad seleccionada.

Se distinguen primitivas, referencias con argumento, parámetros asociados a su declaración y arreglos con tipo componente. La excepción de herencia sin argumento debe conservar el parámetro original. Comparar sólo lexemas no alcanza para genericidad ni arreglos.

Asignabilidad y clasificación como llamada son independientes del tipo. `a.length` y `a[0]` pueden tener el mismo tipo `int`, pero sólo el segundo es asignable. Una llamada `int` puede ser sentencia; un literal `int` no. Una expresión parentizada conserva el tipo, pero no las características estructurales de su contenido.

El chequeo termina antes de generar código. Offsets, disposición de objetos y arreglos, despacho de llamadas e instrucciones de la MV corresponden a Etapa 5. También se deja para esa etapa el control del punto de entrada: en Etapas 3 y 4 no se exige existencia, unicidad ni signatura de `main`.